Fama-French 5-Factor Model: Calculations & Visualizations

Group 2

2026-04-18

Contents

0. Install packages and csv files for chosen assets. **1**

1. Data Ingestion **2**

2. Preprocessing & Normalization **5**

3. Descriptive Statistical Profiling **5**

4. Multivariate Regression Modeling **6**

5. Visual Analytics **7**

6. Model Diagnostics & Significance Audit **13**

Live Dashboard: case1group2finlyts.netlify.app
GitHub Repository: github.com/sakudiff/FINLYTS-Group-2

0. Install packages and csv files for chosen assets.

```
# --- 1. Environment Setup & Dependencies ---
# This section ensures all necessary libraries are installed and loaded.
# We utilize specialized R libraries for data science (tidyverse),
# financial analysis (tidyquant), and statistical reporting (broom).
knitr::opts_chunk$set(
  echo    = TRUE,
  warning = FALSE,
  message = FALSE,
  tidy    = TRUE,
  tidy.opts = list(width.cutoff = 72)
)

required_packages <- c("tidyverse", "tidyquant", "quantmod", "broom",
↪  "PerformanceAnalytics", "tinytex", "moments", "zoo", "ggrepel")
new_packages <- required_packages[!(required_packages %in%
↪  installed.packages()[,"Package"])]

if (length(new_packages)) {
  install.packages(new_packages, repos = "http://cran.us.r-project.org")
}

library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(tidyquant)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
## -- Attaching core tidyquant packages ------------------------ tidyquant 1.0.12 --
## v PerformanceAnalytics 2.0.8      v TTR                 0.24.4
## v quantmod             0.4.28     v xts                 0.14.2-- Conflicts ------------------------
## x zoo::as.Date()                 masks base::as.Date()
## x zoo::as.Date.numeric()         masks base::as.Date.numeric()
## x dplyr::filter()                masks stats::filter()
## x xts::first()                   masks dplyr::first()
## x dplyr::lag()                   masks stats::lag()
## x xts::last()                    masks dplyr::last()
## x PerformanceAnalytics::legend() masks graphics::legend()
## x quantmod::summary()            masks base::summary()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(quantmod)
library(broom)
library(PerformanceAnalytics)
library(moments)
```

```
##
## Attaching package: 'moments'
##
## The following objects are masked from 'package:PerformanceAnalytics':
##
##     kurtosis, skewness
```

```r
library(zoo)
library(ggrepel)
```

1. Data Ingestion

This stage handles the automated acquisition of historical price data and risk factors. As detailed in the methodology, we define a set of USD-denominated tickers to align with the Kenneth French factors. The script fetches asset data from Yahoo Finance and loads the Five-Factor data from the local Kenneth French Data Library, utilizing a local caching mechanism to optimize network traffic.

```r
# 1. Configuration & Tickers
options(download.file.method = "libcurl")
```

```r
library(lubridate)

# Using USD-denominated tickers to align with Fama-French USD Factors
tickers <- c("VUSD.L", "WSML.L", "VDEV.L", "QUAL", "VIG", "XLK", "IAU", "VWRA.L")

if (!dir.exists("data")) {
    dir.create("data")
}

# 2. Automated Data Collection
message("Checking/Updating historical data from Yahoo Finance...")

lapply(tickers, function(symbol) {
    file_path <- paste0("data/", symbol, ".csv")

    if (!file.exists(file_path)) {
        tryCatch({
            df <- suppressWarnings(getSymbols(symbol, src = "yahoo", from = "2015-01-01",
                auto.assign = FALSE))
            if (!is.null(df) && nrow(df) > 0) {
                df_tidy <- as.data.frame(na.omit(df)) %>%
                  rownames_to_column(var = "Date") %>%
                  as_tibble()
                write_csv(df_tidy, file_path)
                message(paste("Downloaded:", symbol))
            }
        }, error = function(e) message(paste("Failed to download", symbol,
            ":", e$message)))
    } else {
        message(paste("Using existing local data for:", symbol))
    }
})
```

```
## [[1]]
## NULL
##
## [[2]]
## NULL
##
## [[3]]
## NULL
##
## [[4]]
## NULL
##
## [[5]]
## NULL
##
## [[6]]
## NULL
##
## [[7]]
## NULL
##
```

```
## [[8]]
## NULL
```

```r
# 3. Monthly Return Calculation Function
load_asset_returns <- function(ticker) {
    file_path <- paste0("data/", ticker, ".csv")
    if (!file.exists(file_path))
        return(NULL)

    df <- read_csv(file_path, show_col_types = FALSE) %>%
        mutate(Date = as.Date(Date)) %>%
        arrange(Date)

    adj_col <- grep("Adjusted", names(df), value = TRUE)
    if (length(adj_col) == 0)
        return(NULL)

    xts_df <- xts(df[[adj_col]], order.by = df$Date)
    monthly_rets <- periodReturn(xts_df, period = "monthly", type = "log")

    df_ret <- as.data.frame(monthly_rets) %>%
        rownames_to_column(var = "Date") %>%
        mutate(Date = as.Date(Date)) %>%
        rename(!!ticker := monthly.returns)

    return(df_ret)
}

# Load assets and merge
asset_data_list <- compact(lapply(tickers, load_asset_returns))

if (length(asset_data_list) == 0) {
    stop("CRITICAL: No asset data found. Check tickers and internet.")
}

asset_returns <- asset_data_list %>%
    reduce(full_join, by = "Date")
active_tickers <- names(asset_returns)[names(asset_returns) != "Date"]

# 4. Load Kenneth French Factors
ff5_raw <- read_csv("data/F-F_Research_Data_5_Factors_2x3.csv", skip = 3,
    show_col_types = FALSE)
monthly_end_row <- which(is.na(ff5_raw[[1]]) | grepl("Annual Factors", ff5_raw[[1]]))[1]
↪  -
    1

ff5_monthly <- ff5_raw[1:monthly_end_row, ] %>%
    rename(Date_Raw = 1) %>%
    mutate(Date = as.Date(paste0(Date_Raw, "01"), format = "%Y%m%d"), across(c(`Mkt-RF`,
        SMB, HML, RMW, CMA, RF), ~as.numeric(.)/100)) %>%
    # Sanitize names: replace dashes with underscores for syntactic
    # safety in OLS
rename(Mkt_RF = `Mkt-RF`) %>%
    select(Date, Mkt_RF, SMB, HML, RMW, CMA, RF)
```

```r
# 5. Temporal Alignment Synchronizing dates to the first of each month
# to ensure perfect factor alignment.
asset_returns <- asset_returns %>%
    mutate(Date = floor_date(Date, "month"))
ff5_monthly <- ff5_monthly %>%
    mutate(Date = floor_date(Date, "month"))
```

2. Preprocessing & Normalization

This section performs the critical task of synchronizing asset returns with factor premiums. We calculate log-returns and derive "Excess Returns" by subtracting the risk-free rate (RF), which allows us to isolate the compensation earned specifically for market risk exposure.

```r
# 1. Merge assets with factors
final_data <- inner_join(asset_returns, ff5_monthly, by = "Date")

# 2. Pivot longer for analysis Pivoting allows for the vectorized
# application of models across all assets.
final_data_long <- final_data %>%
    pivot_longer(cols = all_of(active_tickers), names_to = "Asset", values_to = "Return")
    ↪  %>%
    filter(!is.na(Return)) %>%
    mutate(Excess_Return = Return - RF)

if (nrow(final_data_long) == 0) {
    stop("ERROR: No overlapping data between assets and factors.")
}

# 3. Factor Correlation Auditing the relationship between the five risk
# premiums to detect potential multicollinearity.
factor_corr <- final_data %>%
    select(Mkt_RF, SMB, HML, RMW, CMA) %>%
    cor(use = "pairwise.complete.obs")

# 4. Cumulative Returns (Wealth Index) Calculating the growth of a
# hypothetical $1 investment over the analysis horizon.
cumulative_returns <- asset_returns %>%
    pivot_longer(-Date, names_to = "Asset", values_to = "Return") %>%
    filter(!is.na(Return)) %>%
    group_by(Asset) %>%
    arrange(Date) %>%
    mutate(Cumulative = exp(cumsum(Return))/exp(first(Return))) %>%
    ungroup()
```

3. Descriptive Statistical Profiling

Prior to regression analysis, we perform a distributional audit. This stage computes central tendency (Mean, Median) and dispersion (SD, Range), along with higher-moment metrics (Skewness, Kurtosis) to identify non-normalities and the impact of historical market shocks.

```r
# Descriptive Statistics Table
stats_table <- data.frame(Mean = apply(asset_returns[active_tickers], 2,
    mean, na.rm = TRUE), Median = apply(asset_returns[active_tickers], 2,
```

```r
    median, na.rm = TRUE), SD = apply(asset_returns[active_tickers], 2, sd,
    na.rm = TRUE), Min = apply(asset_returns[active_tickers], 2, min, na.rm = TRUE),
    Max = apply(asset_returns[active_tickers], 2, max, na.rm = TRUE), Skewness =
    ↪  apply(asset_returns[active_tickers],
        2, skewness, na.rm = TRUE), Kurtosis = apply(asset_returns[active_tickers],
        2, kurtosis, na.rm = TRUE))

stats_table
```

	Mean	Median	SD	Min	Max	Skewness	Kurtosis
VUSD.L	0.0100032	0.0162035	0.0405144	-0.1026648	0.0997837	-0.5219081	3.327425
WSML.L	0.0066440	0.0159306	0.0542597	-0.2110441	0.1420327	-0.7211161	4.833064
VDEV.L	0.0085248	0.0161081	0.0398528	-0.1104577	0.1193215	-0.5438626	3.663242
QUAL	0.0098264	0.0115503	0.0436706	-0.1209254	0.1158067	-0.5031619	3.405714
VIG	0.0091052	0.0143036	0.0380480	-0.1008241	0.0956425	-0.3931366	3.283113
XLK	0.0150060	0.0183829	0.0542537	-0.1274597	0.1287142	-0.3025565	2.671648
IAU	0.0106593	0.0034217	0.0419649	-0.0873066	0.1165092	0.2906993	2.759118
VWRA.L	0.0094186	0.0193661	0.0422576	-0.1163591	0.1127406	-0.5805453	3.606279

4. Multivariate Regression Modeling

The core of our econometric analysis involves fitting the Fama-French 5-Factor model using Ordinary Least Squares (OLS) regression. We employ a modern functional programming approach (nest and map) to decompose each asset's excess returns into the five systematic risk premiums: Market, Size, Value, Profitability, and Investment.

```r
# Robust regression workflow using nest/map
model_results <- final_data_long %>%
    group_by(Asset) %>%
    nest() %>%
    mutate(model = map(data, ~lm(Excess_Return ~ Mkt_RF + SMB + HML + RMW +
        CMA, data = .x)), tidied = map(model, tidy), glanced = map(model,
        glance)) %>%
    unnest(tidied) %>%
    select(Asset, term, estimate, std.error, statistic, p.value)

# Extract model fitness (Adjusted R-squared)
model_glance <- final_data_long %>%
    group_by(Asset) %>%
    nest() %>%
    mutate(model = map(data, ~lm(Excess_Return ~ Mkt_RF + SMB + HML + RMW +
        CMA, data = .x)), glanced = map(model, glance)) %>%
    unnest(glanced) %>%
    select(Asset, r.squared, adj.r.squared, p.value)

# Pivot for summary view
regression_summary <- model_results %>%
    select(Asset, term, estimate, p.value) %>%
    pivot_wider(names_from = term, values_from = c(estimate, p.value))

regression_summary
```

Asset	estimate_(Intercept)	estimate_Mkt_RF	estimate_SMB	estimate_HML	estimate_RMW	estimate_CMA	p.value_(Intercept)	p.value_Mkt_RF	p.value_SMB	p.value_HML	p.value_RMW	p.value_CMA
VUSD.L	-0.0002529	0.9106084	0.1025607	-0.0455520	-0.0266915	0.0956504	0.7748654	0.0000000	0.0061639	0.1797724	0.5663789	0.0585449
VDEV.L	-0.0020254	0.8894272	0.0649988	-0.0153942	-0.0300442	0.1184609	0.0473111	0.0000000	0.1251132	0.6910487	0.5728982	0.0409881
QUAL	-0.0019064	0.9824798	0.0923409	-0.0339952	0.1987651	-0.0104927	-0.0110024	0.0000000	0.0032558	0.2306839	0.0000001	0.2802455
VIG	-0.0015954	0.8468753	0.0561170	-0.0221199	0.2560127	0.2073455	0.1230627	0.0000000	0.1921541	0.5744559	0.0000006	0.2000544
XLK	0.0018097	1.1077233	0.2541574	-0.2190831	0.1219808	-0.1726884	-0.3045105	0.0000000	0.0007163	0.0014483	0.1889553	0.0854847
IAU	0.0020384	0.1324289	0.0894536	-0.3380870	0.3466965	0.5232666	0.5741906	0.1190172	0.5538674	0.0160812	0.2071091	0.5012111 6
WSML.L	-0.0046149	0.9070231	0.5217198	0.1136370	-0.0212765	-0.0600156	0.0039675	0.0000000	0.0000000	0.0045117	0.3782876	0.1461977 8
VWRA.L	-0.0029486	0.8511028	0.0047584	-0.0172106	-0.0030540	0.1737220	0.0415366	0.0000000	0.9336846	0.6724318	0.6962281	0.3014364 0

5. Visual Analytics

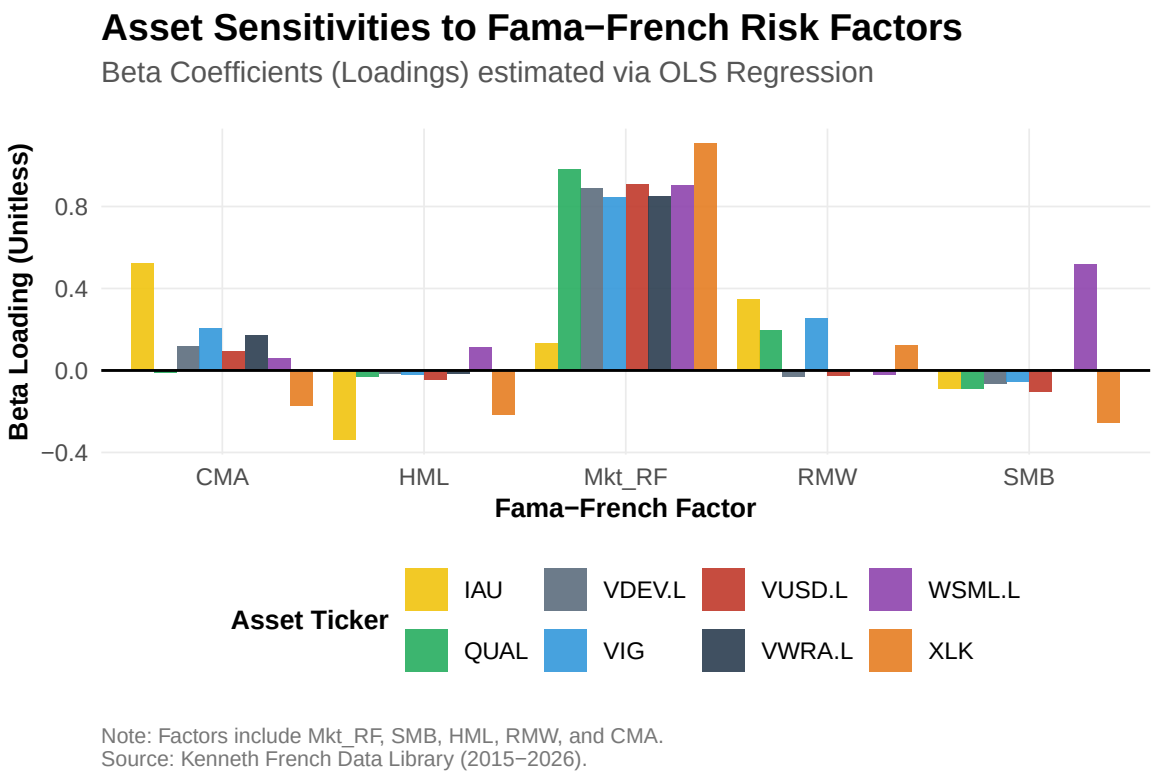
To facilitate interpretation, we generate high-fidelity visualizations. These charts map asset sensitivities (Factor Loadings), evaluate model accuracy (Adjusted R-squared), and trace historical growth trajectories. All plots utilize a consistent color palette and a custom `theme_quant` for professional publication standards.

```
# 0. Consistent Color Palette & Custom Theme
asset_colors <- c(
  "VWRA.L" = "#2C3E50",  # Benchmark (Dark Navy)
  "VDEV.L" = "#5D6D7E",  # Developed
  "WSML.L" = "#8E44AD",  # Small Cap
  "QUAL"   = "#27AE60",  # Quality
  "VIG"    = "#3498DB",  # Dividend
  "XLK"    = "#E67E22",  # Tech
  "IAU"    = "#F1C40F",  # Gold
  "VUSD.L" = "#C0392B"   # S&P 500 (Red)
)

theme_quant <- function() {
  theme_minimal() +
    theme(
      plot.title = element_text(face = "bold", size = 14, margin = margin(b = 5)),
      plot.subtitle = element_text(size = 11, color = "#555555", margin = margin(b =
      ↪  15)),
      plot.caption = element_text(size = 8, color = "#777777", hjust = 0, margin =
      ↪  margin(t = 15)),
      axis.title = element_text(size = 10, face = "bold"),
      axis.text = element_text(size = 9),
      panel.grid.minor = element_blank(),
      panel.grid.major = element_line(color = "#EBEBEB", linewidth = 0.3),
      legend.position = "bottom",
      legend.title = element_text(size = 10, face = "bold"),
      legend.text = element_text(size = 9),
      axis.ticks = element_blank(),
      plot.margin = margin(20, 20, 20, 20)
```

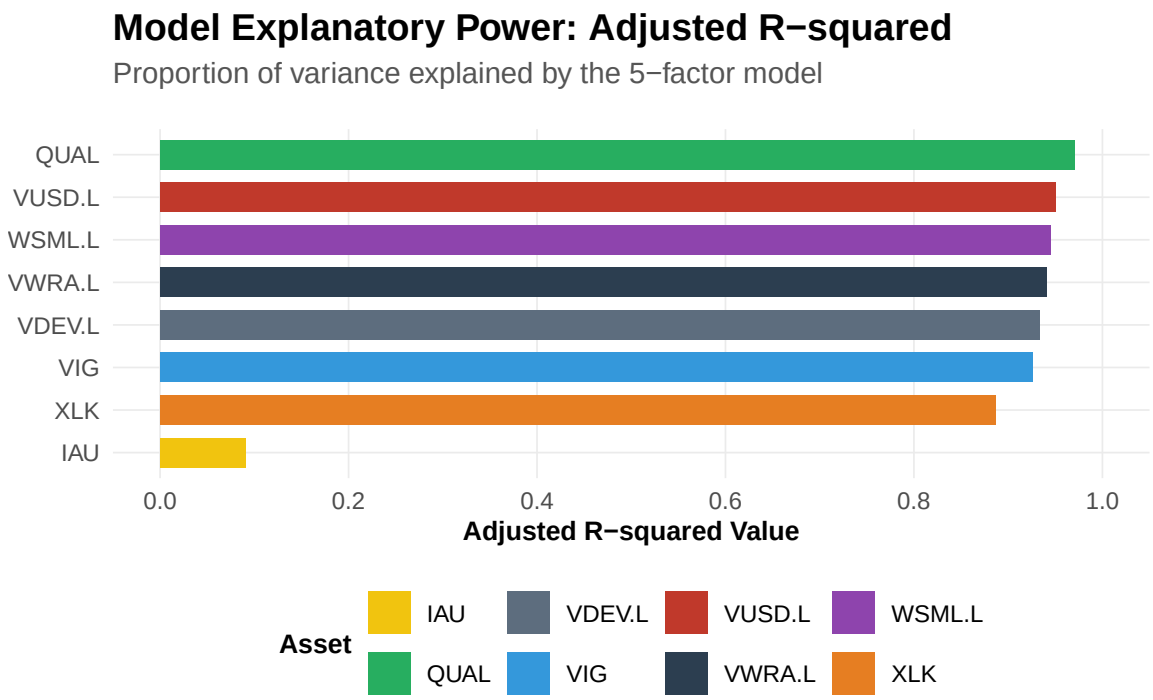
```r
    )
}

# 1. Coefficient Bar Chart (Factor Loadings)
coef_plot <- model_results %>%
  filter(term != "(Intercept)") %>%
  ggplot(aes(x = term, y = estimate, fill = Asset)) +
  geom_bar(stat = "identity", position = "dodge", alpha = 0.9) +
  theme_quant() +
  scale_fill_manual(values = asset_colors) +
  labs(title = "Asset Sensitivities to Fama-French Risk Factors",
       subtitle = "Beta Coefficients (Loadings) estimated via OLS Regression",
       x = "Fama-French Factor",
       y = "Beta Loading (Unitless)",
       fill = "Asset Ticker",
       caption = "Note: Factors include Mkt_RF, SMB, HML, RMW, and CMA.\nSource: Kenneth
       ↪  French Data Library (2015-2026).") +
  geom_hline(yintercept = 0, color = "black", linewidth = 0.5)

# 2. Model Accuracy (Adjusted R-squared)
rsquared_plot <- model_glance %>%
  ggplot(aes(x = reorder(Asset, adj.r.squared), y = adj.r.squared, fill = Asset)) +
  geom_bar(stat = "identity", width = 0.7) +
  coord_flip() +
  theme_quant() +
  scale_fill_manual(values = asset_colors) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(title = "Model Explanatory Power: Adjusted R-squared",
       subtitle = "Proportion of variance explained by the 5-factor model",
       x = NULL,
       y = "Adjusted R-squared Value",
       fill = "Asset",
       caption = "Higher values indicate better model fit.")

# 3. Cumulative Returns Plot
cum_plot <- cumulative_returns %>%
  ggplot(aes(x = Date, y = Cumulative, color = Asset)) +
  geom_line(linewidth = 0.8) +
  theme_quant() +
  scale_y_log10(breaks = c(1, 1.5, 2, 3, 4, 5)) +
  scale_color_manual(values = asset_colors) +
  labs(title = "Wealth Index: Cumulative Growth of $1 Investment",
       subtitle = "Historical Performance (Logarithmic Scale), 2015 - 2026",
       x = "Year",
       y = "Value of $1 (USD)",
       caption = "Source: Yahoo Finance. First date normalized to 1.00.")

# 4. Security Market Line (SML) Scatter Plot
scatter_data <- final_data_long %>%
  group_by(Asset) %>%
  summarise(
    Mean_Excess = mean(Excess_Return, na.rm = TRUE),
    SD = sd(Return, na.rm = TRUE),
    .groups = "drop"
```

```r
  ) %>%
  left_join(model_results %>% filter(term == "Mkt_RF") %>% select(Asset, estimate), by =
  ↪  "Asset")

scatter_plot <- scatter_data %>%
  mutate(Mean_Excess_Pct = Mean_Excess * 100) %>%
  ggplot(aes(x = estimate, y = Mean_Excess_Pct, label = Asset)) +
  geom_smooth(method = "lm", se = FALSE, color = "#CCCCCC", linetype = "dashed",
  ↪  linewidth = 0.5) +
  geom_point(aes(size = SD, color = Asset), alpha = 0.8) +
  ggrepel::geom_text_repel(size = 3, fontface = "bold", box.padding = 0.5) +
  theme_quant() +
  scale_color_manual(values = asset_colors) +
  scale_size_area(max_size = 10) +
  labs(title = "Security Market Line (SML): Risk-Adjusted Returns",
       subtitle = "Mean Monthly Excess Return (%) vs. Market Beta (Loading)",
       x = "Market Beta (beta)",
       y = "Mean Monthly Excess Return (%)",
       size = "Volatility (SD)",
       caption = "Dashed line represents the empirical SML.")

# 5. Rolling Beta Analysis (Stability Diagnostic)
rolling_beta_data <- final_data %>%
  select(Date, VUSD.L, Mkt_RF, RF) %>%
  mutate(VUSD_Excess = VUSD.L - RF)

roll_reg <- function(d) {
  coef(lm(VUSD_Excess ~ Mkt_RF, data = as.data.frame(d)))[2]
}

rolling_beta_data$Beta <- zoo::rollapply(rolling_beta_data %>% select(VUSD_Excess,
↪  Mkt_RF),
                                         width = 24, FUN = roll_reg, by.column = FALSE,
                                         ↪  fill = NA)

rolling_beta_plot <- rolling_beta_data %>%
  filter(!is.na(Beta)) %>%
  ggplot(aes(x = Date, y = Beta)) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "#C0392B", linewidth = 0.8) +
  geom_line(color = "#C0392B", linewidth = 1) +
  theme_quant() +
  labs(title = "Stability Analysis: 24-Month Rolling Market Beta",
       subtitle = "Asset: VUSD.L (S&P 500 ETF) relative to Market Factor",
       x = "Analysis Period",
       y = "Rolling Beta Loading",
       caption = "Red dashed line at 1.00 represents market-neutral sensitivity.")

# Create directories and save plots for Final Paper integration
if (!dir.exists("image/paper")) dir.create("image/paper", recursive = TRUE)
ggsave("image/paper/factor_loadings.png", coef_plot, width = 10, height = 6)
ggsave("image/paper/model_accuracy.png", rsquared_plot, width = 8, height = 5)
ggsave("image/paper/cumulative_returns.png", cum_plot, width = 10, height = 6)
ggsave("image/paper/risk_return_scatter.png", scatter_plot, width = 8, height = 6)
ggsave("image/paper/rolling_beta.png", rolling_beta_plot, width = 10, height = 5)
```

```r
# Display active plots
coef_plot
```

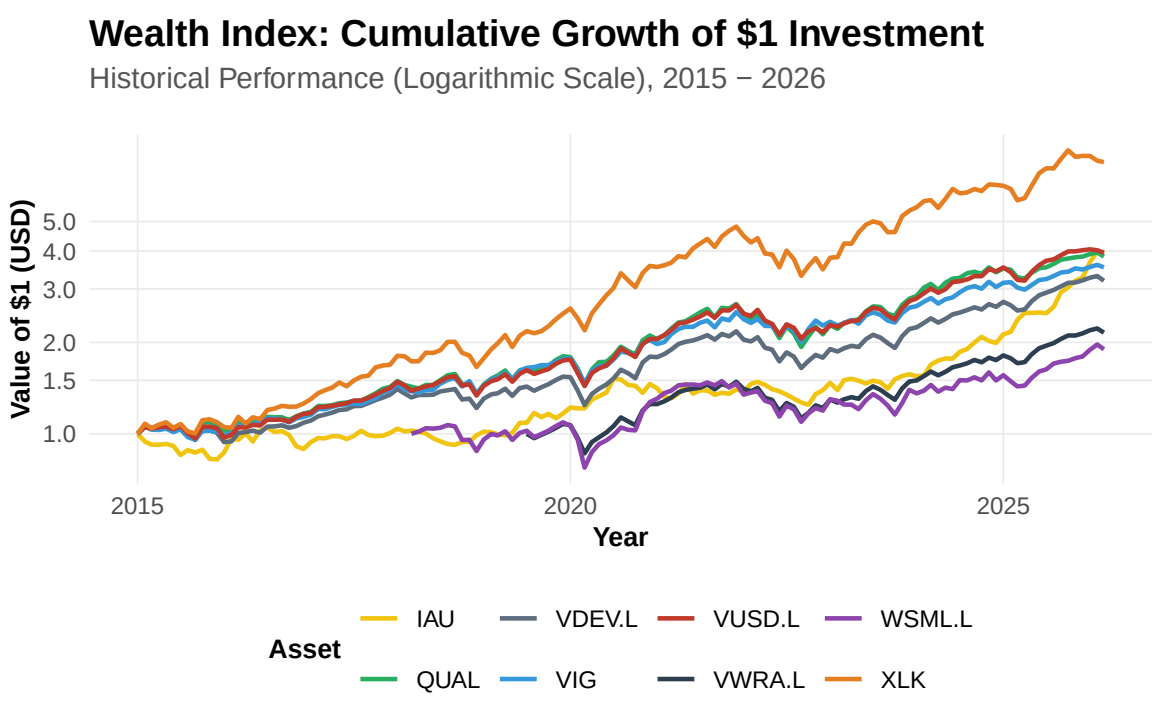



```r
rsquared_plot
```



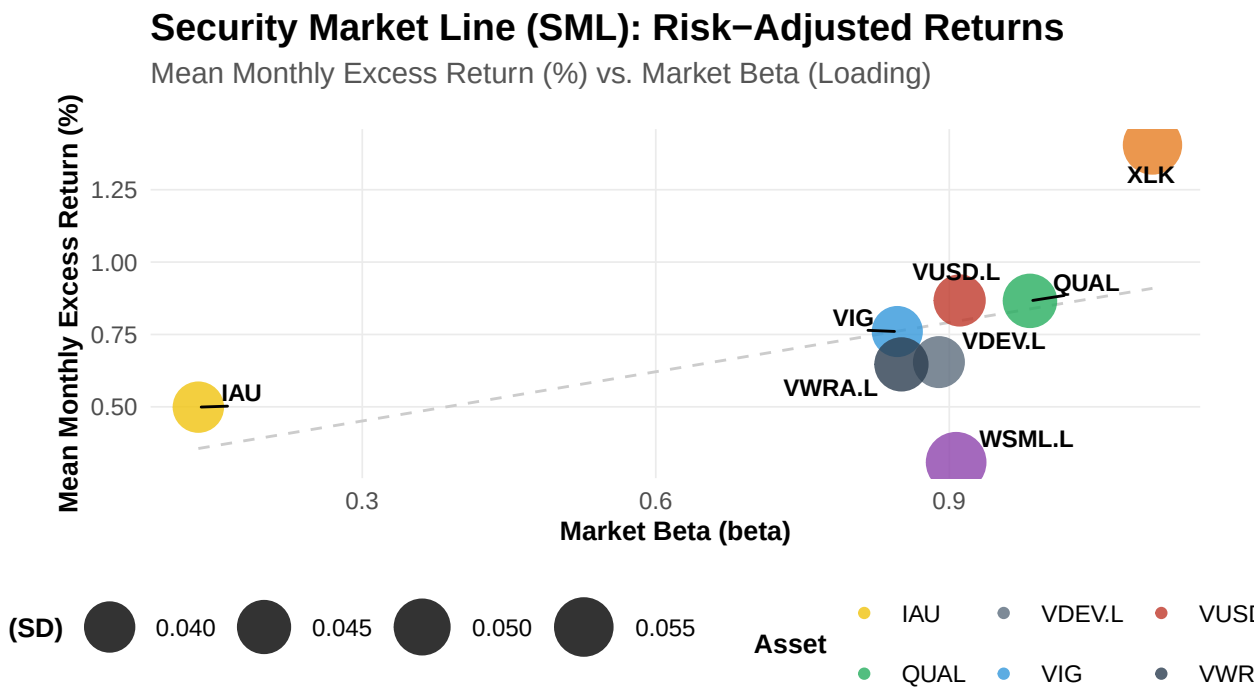
Higher values indicate better model fit.

```
cum_plot
```



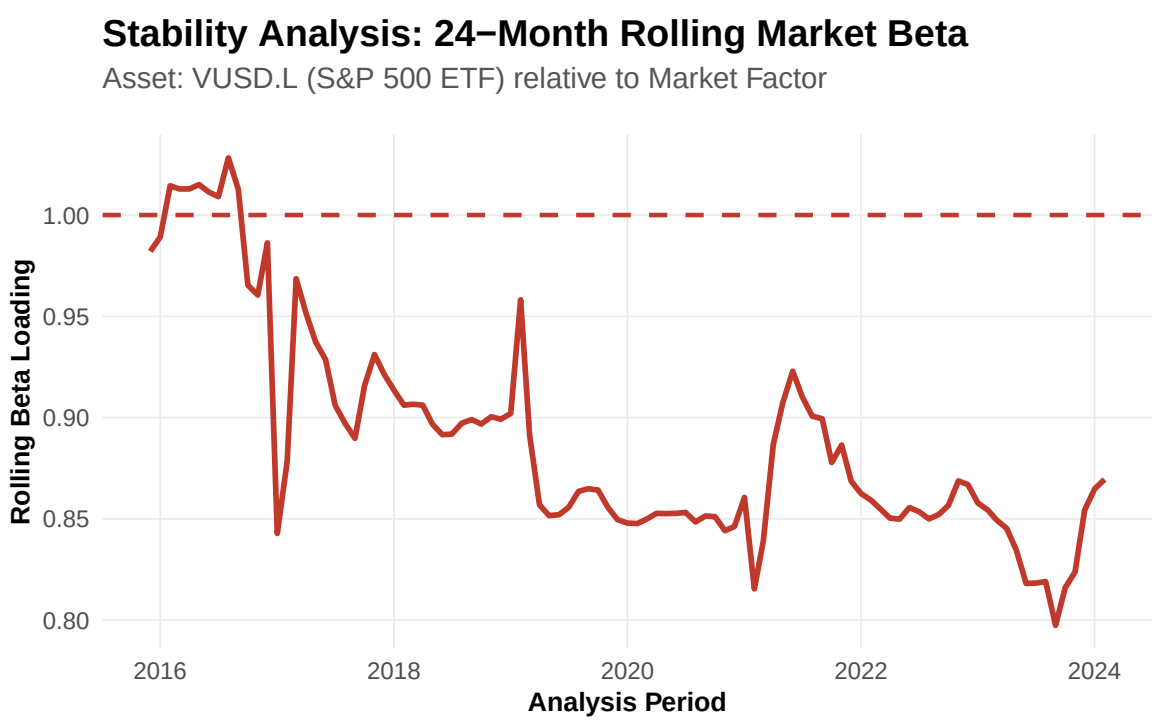
Source: Yahoo Finance. First date normalized to 1.00.

```
scatter_plot
```



Dashed line represents the empirical SML.

```
rolling_beta_plot
```



Red dashed line at 1.00 represents market−neutral sensitivity.

6. Model Diagnostics & Significance Audit

The final stage of the workflow involves a rigorous audit of statistical significance. We calculate p-values for all factor loadings and apply standard academic "significance stars." This diagnostic ensures the reliability of our findings and identifies which risk premiums are the primary drivers of performance across the portfolio.

```r
# 1. Model Fitness Summary (Sorted by Adjusted R-squared)
fitness_summary <- model_glance %>%
    arrange(desc(adj.r.squared)) %>%
    rename(`R-squared` = r.squared, `Adj R-squared` = adj.r.squared, `Model P-value` =
    ↪  p.value)

# 2. Significance Audit (Binary Yes/No at 95% Confidence)
significance_summary <- model_results %>%
    mutate(significant = ifelse(p.value < 0.05, "Yes", "No")) %>%
    select(Asset, term, significant) %>%
    pivot_wider(names_from = term, values_from = significant)

# 3. Formatted Coefficient Table (Stars)
formatted_coeffs <- model_results %>%
    mutate(stars = case_when(p.value < 0.001 ~ "***", p.value < 0.01 ~ "**",
        p.value < 0.05 ~ "*", TRUE ~ ""), estimate_star = paste0(round(estimate,
        4), stars)) %>%
    select(Asset, term, estimate_star) %>%
    pivot_wider(names_from = term, values_from = estimate_star)

# Display Diagnostic Tables
print("--- MODEL FITNESS (ADJUSTED R-SQUARED) ---")
```

```
## [1] "--- MODEL FITNESS (ADJUSTED R-SQUARED) ---"
```

```r
print(fitness_summary)
```

```
## # A tibble: 8 x 4
## # Groups:   Asset [8]
##   Asset  `R-squared` `Adj R-squared` `Model P-value`
##   <chr>        <dbl>           <dbl>           <dbl>
## 1 QUAL         0.972          0.971         8.52e-87
## 2 VUSD.L       0.952          0.950         1.77e-73
## 3 WSML.L       0.949          0.945         1.81e-47
## 4 VWRA.L       0.945          0.941         1.55e-36
## 5 VDEV.L       0.936          0.933         2.69e-66
## 6 VIG          0.929          0.926         1.01e-63
## 7 XLK          0.891          0.886         4.00e-53
## 8 IAU          0.129          0.0905        7.12e- 3
```

```r
print("--- FACTOR SIGNIFICANCE AUDIT (P < 0.05) ---")
```

```
## [1] "--- FACTOR SIGNIFICANCE AUDIT (P < 0.05) ---"
```

```r
print(significance_summary)
```

```
## # A tibble: 8 x 7
## # Groups:   Asset [8]
##   Asset  `(Intercept)` Mkt_RF SMB   HML   RMW   CMA
##   <chr>  <chr>         <chr>  <chr> <chr> <chr> <chr>
## 1 VUSD.L No            Yes    Yes   No    No    No
```

```
## 2 VDEV.L Yes           Yes    No    No    No    Yes
## 3 QUAL   Yes           Yes    Yes   No    Yes   No
## 4 VIG    No            Yes    No    No    Yes   Yes
## 5 XLK    No            Yes    Yes   Yes   No    No
## 6 IAU    No            No     No    Yes   No    Yes
## 7 WSML.L Yes           Yes    Yes   Yes   No    No
## 8 VWRA.L Yes           Yes    No    No    No    Yes
```

```r
print("--- REGRESSION COEFFICIENTS WITH SIGNIFICANCE STARS ---")
```

```
## [1] "--- REGRESSION COEFFICIENTS WITH SIGNIFICANCE STARS ---"
```

```r
print(formatted_coeffs)
```

```
## # A tibble: 8 x 7
## # Groups:   Asset [8]
##   Asset  `(Intercept)` Mkt_RF    SMB        HML       RMW       CMA
##   <chr>  <chr>         <chr>     <chr>      <chr>     <chr>     <chr>
## 1 VUSD.L -3e-04        0.9106*** -0.1026**  -0.0456   -0.0267   0.0957
## 2 VDEV.L -0.002*       0.8894*** -0.065     -0.0154   -0.03     0.1185*
## 3 QUAL   -0.0019*      0.9825*** -0.0923**  -0.034    0.1988*** -0.0105
## 4 VIG    -0.0016       0.8469*** -0.0561    -0.0221   0.256***  0.2073***
## 5 XLK    0.0018        1.1077*** -0.2542*** -0.2191** 0.122     -0.1727
## 6 IAU    0.002         0.1324    -0.0895    -0.3381*  0.3467    0.5233*
## 7 WSML.L -0.0046**     0.907***  0.5217***  0.1136*   -0.0213   0.06
## 8 VWRA.L -0.0029*      0.8511*** 0.0048     -0.0172   -0.0031   0.1737*
```